

Redis Streams

내부 구조

Append-Only Log · Consumer Group · PEL · At-least-once
이론 전용 강의 55분 — RedisStreamPublisher 코드 완전 해부

Pub/Sub vs Streams vs Kafka

PEL + XACK + XAUTOCLAIM

실습 2 파트 포함

COINCRAFT
ACADEMY

왜 실습은 Redis Streams인가 — Kafka와의 관계

SAME CONCEPTS, LOWER FRICTION — REPLACE ONLY THE IMPLEMENTATION

교보생명 실제 운영 환경은 **Kafka**다. 그런데 이 강의 실습은 **Redis Streams**로 진행한다.

항목	Redis Streams	Kafka
실습 환경 세팅	<code>docker-compose up redis</code> 한 줄	ZooKeeper/KRaft + Broker 별도 설치
핵심 개념	Consumer Group, At-least-once, PEL, ACK	동일 (Consumer Group, offset commit)
처리량	수만 msg/초 (강의 실습 충분)	수백만 msg/초 (운영 필요)
메시지 보존	메모리 기반, TTL/MAXLEN	디스크, 무제한 보존
운영 복잡도	낮음	높음

핵심: 개념이 동일하다. Redis Streams로 배운 Consumer Group, 멍등성, DLQ, 재처리 패턴은 Kafka에 그대로 적용된다.

운영 전환 시: RedisStreamPublisher와 ConsumerGroupWorker만 Kafka 구현체로 교체 — 비즈니스 로직은 건드리지 않는다.

Redis Streams ↔ Kafka 명령어 대응표

API MAPPING — IDENTICAL INTENT, DIFFERENT SYNTAX

Redis Streams	Kafka	공통 의미
XADD	produce()	메시지를 로그에 발행
XREADGROUP	poll()	Consumer Group으로 메시지 수신 (소유권 등록)
XACK	commitOffset()	처리 완료 선언
PEL (Pending Entry List)	__consumer_offsets	처리 완료 미확인 메시지 추적
XAUTOCLAIM	재조인 후 미commit offset 재수신	Consumer 장애 후 미처리 메시지 복구
XGROUP CREATE	Consumer Group 등록	협력 처리 그룹 생성

운영 전환 시 변경 대상: RedisStreamPublisher → Kafka Producer 구현체 · ConsumerGroupWorker → Kafka Consumer 구현체
비즈니스 로직(LedgerService, AuditLog 등)은 변경 없음.

이 세션이 답하는 질문

THREE KEY QUESTIONS — STREAMS, AT-LEAST-ONCE, PEL

Q1. Pub/Sub이 있는데 왜 Streams인가?

Pub/Sub은 구독자가 없으면 메시지가 사라진다. 내구성 없음.

Streams는 append-only 로그 — 구독자가 없어도 데이터가 남는다.

→ Redis Streams는 Kafka의 Consumer Group 개념을 Redis에서 구현한 것이다.

Q2. At-least-once란 정확히 무슨 뜻인가?

"최소 한 번은 반드시 처리된다"는 보장.

네트워크 장애나 Consumer crash 후 재시작해도 메시지가 재전달된다.

단, 두 번 처리될 수도 있다(at-least, not exactly-once). 멍등성 필수.

Q3. PEL이란 무엇이고, XACK는 왜 필수인가?

PEL(Pending Entry List) = "읽었지만 아직 처리 완료 확인 안 된 메시지 목록"

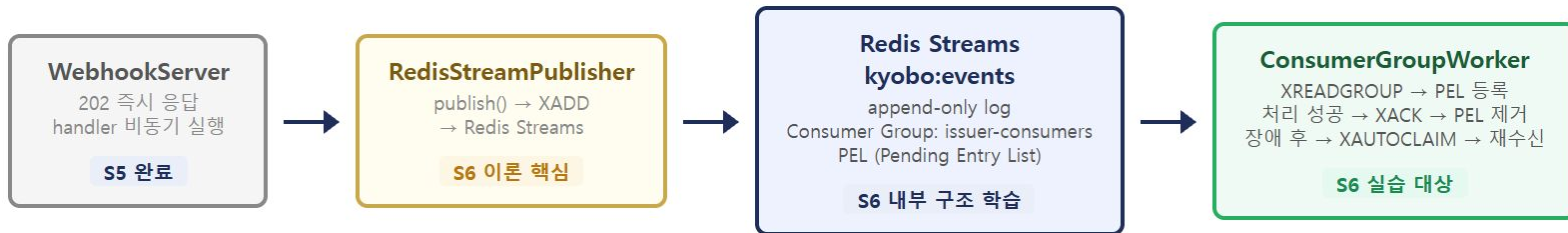
XREADGROUP으로 읽으면 자동으로 PEL에 등록된다.

XACK를 보내야 PEL에서 제거된다 = "처리 완료" 선언.

XACK 없이 Consumer가 crash → PEL에 남음 → 재시작 후 재수신.

전체 흐름에서의 위치

PIPELINE POSITION — S6 FOCUSES ON REDIS STREAMS INTERNALS



이 세션 범위: RedisStreamPublisher(XADD 발행) + Redis Streams 내부 구조 (Entry ID, Consumer Group, PEL) + ConsumerGroupWorker 기본 흐름

1부 — 패턴 1: Pub/Sub (PUBLISH/SUBSCRIBE)

FIRE-AND-FORGET — NO DURABILITY, NOT SUITABLE FOR FINANCE

1부 · Pub/Sub vs Queue vs Streams 비교 (10분)

발행자

구독자A

구독자B

정상 케이스

PUBLISH "msg" → 구독자A 수신 ✓

PUBLISH "msg" → 구독자B 수신 ✓

구독자A 오프라인 케이스

PUBLISH "msg" → 구독자A ✗ 오프라인 → 유실

PUBLISH "msg" → 구독자B 수신 ✓

→ 오프라인 구독자는 메시지를 영구 유실 ✗

특징 & 금융 시스템 판정

- Fire-and-forget: 발행 즉시 메시지 삭제
- 구독자가 없으면 메시지 소멸
- 처리 확인(ACK) 메커니즘 없음
- 오프라인 수신 불가
- Consumer 장애 복구 불가

금융 시스템에서 사용 불가

이벤트 유실 = 회계 불일치

1부 — 패턴 2: 단순 List/Queue (LPUSH/RPOP)

POP = DELETE — NO RECOVERY IF CONSUMER CRASHES



특징

- Consumer가 pop하면 데이터 즉시 삭제
- 처리 중 crash → 재처리 불가
- 처리 완료 확인 메커니즘 없음
- 오프라인 수신은 가능 (큐에 남음)

패턴	내구성	처리 확인	장애 복구
Pub/Sub	없음	없음	불가
List Queue	처리 전까지	없음	불가

at-most-once 보장 — 최대 한 번. 처리 실패 시 메시지 유실. 금융 시스템 사용 불가.

1부 — 패턴 3: Redis Streams (XADD/XREADGROUP/XACK)

APPEND-ONLY LOG + PEL = AT-LEAST-ONCE GUARANTEE

발행자	Redis Stream	Consumer Group
XADD "msg" → 로그에 보존 (삭제 안 됨) XREADGROUP → msg 처리 중 (PEL에 등록)		
Consumer Crash [crash!] XACK 없음 → PEL에 여전히 남아있음		
재시작 후 자동 복구 XAUTOCLAIM → msg 재수신 ✓ (PEL 소유권 이전)		

특징 — Pub/Sub · Queue 대비

- append-only log: 읽어도 삭제되지 않음
- Consumer Group + PEL로 처리 완료 추적
- Consumer crash 후 미처리 메시지 자동 재전달
- 수평 확장: Consumer N개가 메시지 분담
- MAXLEN 설정으로 스트림 크기 제어 가능

at-least-once 보장

금융 이벤트 처리에 적합 ✓

1부 — 패턴 4: Kafka (교보 운영 환경) + 전체 비교표

KAFKA IS THE PRODUCTION TARGET — SAME PATTERN, MORE SCALE

특성	Pub/Sub	List Queue	Redis Streams	Kafka (교보 운영)
메시지 내구성	없음	처리 전까지	MAXLEN 가능	디스크 영구 보존
처리 확인	없음	없음	XACK (PEL)	offset commit
Consumer 장애 복구	불가	불가	XAUTOCLAIM	재조인 후 재수신
수평 확장	모두 수신	1개만 pop	Consumer Group	Consumer Group
처리량	낮음	낮음	수만 msg/초	수백만 msg/초
운영 복잡도	낮음	낮음	낮음	높음
금융 시스템 적합성	✗	△	✓ 강의 실습	✓ 교보 운영

결론: Redis Streams만이 At-least-once를 보장한다. 핵심 개념은 Kafka와 동일 — 운영 전환 시 구현체만 교체.

2부 — Stream Entry 구조

APPEND-ONLY LOG · EACH ITEM IS AN "ENTRY" WITH AUTO-GENERATED ID

2부 · Stream Entry 구조와 XADD (10분)

Stream: kyobo:events — append-only log

Entry #1

ID 1714000000000-0

event Type **NFT_ISSUED**

payload {"tokenId": "42", "owner": "0xABCD"}

txHash 0xdeadbeef...

blockNumber 18500000

requestId req-001

Entry #2

ID 1714000001000-0

event Type **NFT_BURNED**

payload {"tokenId": "43"}

txHash 0xcafebabe...

blockNumber 18500001

requestId req-002

모든 값은 문자열(string)이다: Redis Streams는 string → string 맵만 저장 가능.

숫자(blockNumber) → String() · 객체(payload) → JSON.stringify() 필수. 잊으면 런타임 에러.

Entry ID 형식: {timestamp}-{sequence}

AUTO-GENERATED · TIME-ORDERED · QUERYABLE BY TIME RANGE

1714000000000 - 0

Unix Timestamp (밀리초)

시스템 시계 기반 자동 생성

JS: Date.now()

Python: int(time.time() * 1000)

Sequence 번호

같은 밀리초 내 순서 보장

첫 번째: 0, 두 번째: 1, ...

동시성 충돌 없이 순서 확정

특징

- Redis가 자동 생성 (XADD stream * ...에서 * = 자동)
- 시간 순 정렬 보장
- 같은 밀리초 내에서도 sequence로 순서 보장
- ID로 특정 시점 이후 메시지 범위 조회 가능
- 반환 값: "1714000000000-0" (문자열)

ID 심볼 의미

ID	의미	사용 시점
*	자동 생성	XADD — 일반 발행
\$	현재 시점 이후	XGROUP CREATE
0	처음부터	PEL 재수신
- / +	최솟값 / 최댓값	XRANGE 전체 조회
>	미처리 새 메시지	XREADGROUP 일반 루프

XADD 명령어 — 메시지 발행

APPEND TO STREAM · AUTO ID · MAXLEN TRIM

기본 형식

```
XADD <stream-key> <id> <field1> <value1> [<field2> <value2> ...]
```

id = * : Redis가 자동 생성

```
XADD kyobo:events * \  
  eventType NFT_ISSUED \  
  payload '{"tokenId":"42"}' \  
  txHash 0xdeadbeef \  
  blockNumber 18500000 \  
  requestId req-001 \  
  publishedAt 1714000000000
```

결과: "1714000000000-0" ← 생성된 messageId 반환

크기 제한 (MAXLEN) — 스트림이 무한 증가하지 않도록

```
XADD kyobo:events MAXLEN ~ 10000 * \  
  eventType NFT_ISSUED ...
```

~ : approximate (정확히 10000이 아닐 수 있지만 성능상 유리)

string-string map 제약: blockNumber: 18500000 (number) → 저장 불가 | blockNumber: "18500000" (string) → 저장 가능

XADD 명령어 — 실행 흐름

CLIENT SENDS XADD → REDIS APPENDS ENTRY → AUTO ID RETURNED → MAXLEN TRIM

① XADD kyobo:events * field1 val1 ...

* = ID 자동 생성 지시 · field-value 쌍 전달



② Redis: ID 생성 — {timestamp}-{seq}

Date.now() ms 단위 · 같은 ms이면 seq++ · 단조 증가 보장



③ append-only log에 엔트리 추가

읽어도 삭제 안 됨 · 여러 Consumer가 독립적으로 읽기 가능



④-A MAXLEN 없음

스트림 무한 증가 (개발/테스트용)

④-B MAXLEN ~ 10000

오래된 엔트리 자동 trim (운영 필수)



⑤ "1714000000000-0" 반환 (messageId)

클라이언트가 이 ID를 DB에 저장 → 재처리 추적 기반

string-string 제약

모든 field, value → 문자열만 허용

number → String() 변환 필수

object → JSON.stringify() 필수

MAXLEN ~ vs 정확

MAXLEN ~ N: 근사값, 성능 우선

MAXLEN N: 정확, 매 XADD마다 trim

핵심: XADD는 ID 자동생성 + append-only 적재 + messageId 반환의 세 가지를 한 번에 수행한다. MAXLEN ~ 설정으로 메모리를 제어해야 한다.

RedisStreamPublisher.publish() 코드 분석

INTERFACE DEFINITION · STRING CONVERSION · MESSAGE ID RETURN

```
export interface StreamEvent {
  streamKey: string;           // "kyobo:events"
  eventType: string;           // "NFT_ISSUED", "NFT_BURNED"
  payload: Record<string, unknown>;
  txHash: string;
  blockNumber: number;
  requestId: string;           // Idempotency key
}

async publish(event: StreamEvent): Promise<string> {
  const messageId = await this.redis.xadd(
    event.streamKey ?? this.defaultStream,
    {
      eventType: event.eventType,
      payload: JSON.stringify(event.payload), // ← 객체는 JSON 직렬화 필수
      txHash: event.txHash,
      blockNumber: String(event.blockNumber), // ← 숫자는 string으로 변환
      requestId: event.requestId,
      publishedAt: String(Date.now()),
    },
  );
  return messageId; // "1714000000000-0" - DB에 기록해 At-least-once 추적
}
```

RedisStreamPublisher.publish() — 실행 흐름

INPUT → TYPE COERCION → XADD → MESSAGE ID RETURN

① publish(event: StreamEvent) 호출

streamKey / eventType / payload(object) / txHash / blockNumber(number) / requestId



② 타입 변환 (string-string map 제약)

payload: object (저장 불가)

JSON.stringify() → string ✓

blockNumber: number (저장 불가)

String(blockNumber) ✓



③ redis.xadd(streamKey ?? defaultStream, fields)

id='*' 자동생성 · Redis append-only log에 적재



④ return messageId ("1714000000000-0")

호출자가 DB에 저장 → At-least-once 추적용 키

StreamEvent 인터페이스

streamKey: string (기본값: defaultStream)

eventType: string

payload: Record<string,unknown> → JSON 직렬화

txHash: string

blockNumber: number → String() 변환

requestId: string (먹등성 키)

?? 연산자: streamKey가 null/undefined이면 this.defaultStream 사용

publishedAt: String(Date.now()) — 발행 시각 ms

핵심: Redis의 string-string 제약 때문에 object→JSON, number→String 변환이 필수다. 이를 빠뜨리면 런타임 에러가 발생한다.

TS 문법 — interface · Record · Promise<T>

COMPILE-TIME TYPE TOOLS — RUNTIME HAS NO TRACE

```
export interface StreamEvent { ... }
```

- **interface** = 객체의 타입 형태(shape) 정의
- 클래스와 달리 런타임에 존재 안 함
- 컴파일 타임에 타입 체크만 하고 사라짐
- export 붙으면 다른 파일에서 import 가능

```
Record<string, unknown>
```

- Record<K, V> = 키는 K, 값은 V 타입인 객체 단축 표현
- { [key: string]: unknown }과 같은 의미
- **unknown** = any보다 안전 — 사용 전 타입 좁히기 강제

```
Promise<string>, Promise<void>
```

- Promise<T> = "나중에 T 타입 값을 내놓는 비동기 작업"
- <string> = 결과로 문자열 줌
- <void> = 결과 없음 (성공/실패만 의미)
- 제네릭 <>는 "타입 매개변수" — 함수의 인자처럼 타입에도 인자

```
private readonly redis: RedisStreamClient
```

- **private** = 클래스 외부에서 접근 불가
- **readonly** = 한 번 할당 후 변경 불가
- 생성자 매개변수에 직접 쓰면 필드 자동 등록 (TS 단축 문법)

TS 문법 — ?? · catch(err: unknown) · as re-export

NULLISH COALESCING · SAFE ERROR TYPE · ALIAS EXPORT

```
event.streamKey ?? this.defaultStream
```

- ?? = nullish coalescing 연산자
- 왼쪽이 null 또는 undefined면 오른쪽 사용
- ||와 차이: ||은 0, '', false도 falsy 취급

??는 오직 null/undefined만

```
catch (err: unknown)
```

- 에러 객체 타입이 unknown (TS 4.4+ 기본값)
- 사용 전에 타입 확인 필요 (String(err)로 강제 문자열화)
- catch 블록에서 에러 객체 불필요 → } catch { ... } 괄호 생략 (TS 4.0+)

```
export { RedisStreamPublisher as QueueService }
```

- **alias re-export** — 같은 클래스를 다른 이름으로도 내보냄
- 다른 파일에서 import { QueueService }로 가져오면 동일 클래스
- 커리큘럼 명칭과 실제 구현 명칭이 다를 때 호환성 유지

생성자 단축 문법

```
// 단축형 (TS 전용)
constructor(private readonly redis: RedisStreamClient) {}

// 풀어 쓰면
private readonly redis: RedisStreamClient;
constructor(redis: RedisStreamClient) {
  this.redis = redis;
}
```

initialize() — 기동 시 1회 · BUSYGROUP 무시 패턴

IDEMPOTENT STARTUP — SAFE TO CALL ON EVERY RESTART

```
async initialize(groupName = 'issuer-consumers'): Promise<void> {
  try {
    await this.redis.xgroupCreate(this.defaultStream, groupName, '$', true);
    //
    //
    //
  } catch (err: unknown) {
    // BUSYGROUP = 이미 존재하는 그룹 → 정상 (서비스 재시작 시 매번 호출)
    if (!String(err).includes('BUSYGROUP')) throw err;
    // 다른 에러(연결 실패 등)는 그대로 throw
  }
}
```

┌ ─┐
| |
└───┘ mkstream: true (스트림 없으면 자동 생성)
└───┘ '\$': 지금 이후 새 메시지부터 (과거 무시)

\$ vs 0 — 시작 ID 차이

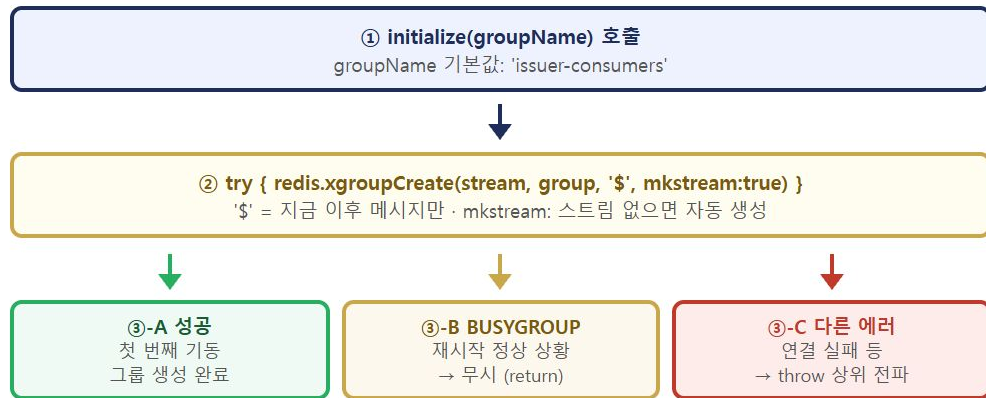
- \$ → 그룹 생성 시점 이후 메시지만 수신 (기존 메시지 무시)
- 0 → 스트림의 처음부터 모든 메시지 수신 (재처리 시 사용)

BUSYGROUP를 무시하는 이유

- 서비스 시작 시 initialize()를 항상 호출
- 처음 실행: XGROUP CREATE 성공
- 재시작 후: BUSYGROUP 에러 → 정상 상황 → 무시
- 다른 에러(연결 실패)는 throw해서 상위 전파
- 결과: **idempotent** — 여러 번 호출해도 안전

initialize() — 실행 흐름

IDEMPOTENT STARTUP: XGROUP CREATE → BUSYGROUP CHECK → SELECTIVE THROW



\$ vs 0 시작 ID

'\$' → 생성 시점 이후 메시지만

'0' → 스트림 처음부터 전부

Idempotent 보장 이유

서비스 시작마다 initialize() 호출

BUSYGROUP 무시 → 재시작 안전

다른 예러 throw → 진짜 문제 감지

핵심: BUSYGROUP만 선택적으로 무시함으로써 여러 번 호출해도 안전한 idempotent 초기화 패턴을 구현한다.

ping() · RedisStreamPublisher 전체 역할 요약

HEALTH CHECK PATTERN · THE BIG PICTURE OF THE CLASS

ping() — 헬스체크

```
async ping(): Promise<boolean> {  
  try {  
    await this.redis.ping();  
    return true;  
  } catch {  
    return false; // throw 안 함 → boolean 반환 (헬스체크 엔드포인트용)  
  }  
}
```

실패해도 throw 안 함 → boolean으로 반환
헬스체크 엔드포인트에서 서비스 상태 판단용

한 줄 요약

블록체인에서 감지한 NFT 이벤트를 **Redis Streams 큐에 발행**해서, 무거운 후속 처리(DB 기록, 웹hook 알림)를 별도 워커가 비동기로 받아 처리하게 만드는 **메시지 브로커 발행자**.

202 패턴이 필요한 이유

- 체인 이벤트 처리가 느리면 → 다음 블록 구독이 밀림 → 이벤트 누락
- "받았다(202 Accepted)"만 즉시 응답 → 큐에 던지고 → 워커가 처리
- 체인 구독은 항상 빠르게 흐름 유지
- publish() 자체는 Redis에 적재만 하고 즉시 리턴 → 200ms 안에 끝남

ping() · RedisStreamPublisher — 실행 흐름

HEALTH CHECK: TRY PING → TRUE / CATCH ANY ERROR → FALSE (NO THROW)

ping() 실행 흐름



헬스체크는 예외를 삼키고 **boolean**으로 통신 — 상위에서 HTTP 200/503 결정

핵심: ping()은 예외를 삼키고 **boolean**을 반환한다. 헬스체크는 예외가 아닌 상태값으로 통신한다.

RedisStreamPublisher 클래스 역할 요약

initialize() — 기동 1회: Consumer Group 생성 (idempotent)

publish() — 이벤트 직렬화 후 XADD → **messageId** 반환

ping() — 연결 상태 **boolean** 반환 (예외 삼킴)

블록체인 이벤트를 Redis Streams에 발행하여 후속 처리 워크가 비동기로 소비하게 하는 **메시지 브로커 발행자**

3부 — Consumer Group이란?

COOPERATIVE PROCESSING — MESSAGES DISTRIBUTED ACROSS WORKERS

3부 · Consumer Group과 PEL 내부 원리 (20분)

Stream: kyobo:events

msg-001 NFT_ISSUED
msg-002 NFT_BURNED
msg-003 NFT_ISSUED
msg-004 NFT_ISSUED

Consumer Group: issuer-consumers

consumer-1

msg-001 처리 중
msg-003 대기 중

consumer-2

msg-002 처리 중
msg-004 대기 중

→ 4개 메시지를 2개 Consumer가 분담 처리

→ Consumer 4개로 늘리면 처리량 2배 증가

Consumer Group의 3가지 장점

- 수평 확장: Consumer 추가만으로 처리량 증가 (코드 변경 0)
- 장애 격리: 한 워커가 죽어도 나머지가 처리 계속
- 자동 인계: 죽은 워커의 PEL은 XAUTOCLAIM으로 다른 워커가 인수

XGROUP CREATE 명령

```
XGROUP CREATE kyobo:events issuer-consumers $ MKSTREAM
```

```
#  
#  
#
```

┌ ── 스트림 없으면 자동 생성
└ ── "\$" = 지금 이후 새 메시지만

PEL (Pending Entry List) 완전 이해

OWNERSHIP TRACKING — THE HEART OF AT-LEAST-ONCE

PEL은 "소유권(ownership)" 추적 장치다. — 읽었지만 아직 XACK 안 한 메시지 목록

Consumer Group: issuer-consumers — PEL 상태

스트림 메시지	PEL (소유권 추적)
msg-001 NFT_ISSUED msg-002 NFT_BURNED	consumer-1 소유 · 미처리 consumer-2 소유 · 미처리
msg-003 NFT_ISSUED msg-004 NFT_ISSUED	(미읽음 — PEL에 없음) (미읽음 — PEL에 없음)

PEL 각 항목에 기록되는 정보

필드	의미
message-id	해당 메시지의 Entry ID
consumer-name	현재 소유 Consumer
idle-time (ms)	마지막 배달 후 경과 시간 → XAUTOCLAIM 기준
delivery-count	총 배달 횟수 → 2 이상이면 재시도된 메시지

delivery-count 활용:

delivery-count >= 최대재시도횟수 → DLQ로 이동

delivery-count < 최대재시도횟수 → 재처리 계속

XREADGROUP과 > 심볼의 의미

NEW MESSAGES vs PEL REDELIVERY — TWO MODES OF XREADGROUP

```
XREADGROUP GROUP issuer-consumers consumer-1 COUNT 10 BLOCK 5000 STREAMS kyobo:events >
```

```
#
```

```
#
```

└ ">" = 미처리 새 메시지만 읽기

```
# ">" 가 아닌 경우:
```

```
XREADGROUP GROUP issuer-consumers consumer-1 COUNT 10 STREAMS kyobo:events 0
```

```
#
```

```
#
```

└ "0" = 내 PEL에 있는 기존 미처리 메시지 재읽기

ID	의미	사용 시점
>	아직 어떤 Consumer에게도 배달 안 된 새 메시지	일반 처리 루프
0	나(this consumer)의 PEL에 있는 모든 미처리 메시지	재시작 후 미처리 재수신
0-0	PEL의 가장 처음부터	XAUTOCLAIM 시작점

BLOCK 옵션: 새 메시지가 없을 때 최대 5000ms(5초) 대기 후 빈 결과 반환 → 바쁜 대기(busy-wait) 없이 효율적인 처리 루프

정상 처리 흐름 — XREADGROUP → XACK

HAPPY PATH: READ → PROCESS → ACK → PEL CLEARED

Step 1: XREADGROUP >

```
XREADGROUP GROUP issuer-consumers consumer-1 COUNT 1 STREAMS kyobo:events >
```

→ msg-001 반환

→ PEL: { msg-001: consumer-1, idleTime:0, deliveryCount:1 }



Step 2: 비즈니스 처리

consumer-1이 msg-001 처리 (LedgerService.update())



Step 3: XACK

```
XACK kyobo:events issuer-consumers msg-001
```

→ PEL에서 msg-001 제거

→ PEL: {} (비어있음) ✓

잘못된 XACK 시점 — 처리 시작 전 ACK

XACK → 처리 중 crash → 이벤트 유실 (PEL에서 이미 제거됨)

올바른 XACK 시점 — 처리 성공 확인 후 ACK

```
try {  
    await processor.process(message);  
    await redis.xack(streamKey, groupName, message.id);  
    // 성공 후 ACK  
} catch {  
    // ACK 안 함 → PEL에 남음 → 재시도  
}
```

정상 처리 흐름 — 실행 흐름

XREADGROUP DELIVERS → PEL RECORDS OWNERSHIP → XACK CLEARS

① XREADGROUP GROUP ... >

미처리 새 메시지 요청 · BLOCK 5000ms 대기



② PEL 등록

{ msg-001: consumer-1, idleTime:0, deliveryCount:1 }



③ processor.process(message)

LedgerService.update() 등 비즈니스 로직 실행



④ XACK → PEL 제거

PEL: {} — 처리 완료 확정 ✓

PEL 상태 변화

Step 1 후: **PENDING** (deliveryCount=1)

Step 3 중: 계속 PENDING (idleTime 증가)

Step 4 후: **제거** → 완료

Step 3에서 crash 발생 시

XACK 없음 → PEL에 잔류

idle 초과 → XAUTOCLAIM → 재배달

deliveryCount: 1 → 2

핵심: XREADGROUP이 PEL에 소유권을 등록하고, XACK만이 그것을 제거한다. 이 두 명령이 At-least-once의 전부다.

장애 복구 흐름 — XAUTOCLAIM

CONSUMER CRASH RECOVERY: PEL IDLE TIMEOUT → OWNERSHIP TRANSFER

1

XREADGROUP ... >
msg-002 반환 · PEL: consumer-1 | idleTime: 0

2

consumer-1 [crash!]
XACK 없음 → PEL에 msg-002 그대로 남음

3

⌚ 30,000ms 경과 — minIdleMs 초과

4

consumer-2 XAUTOCLAIM 호출
msg-002 발견 → 소유권 이전: consumer-2 | deliveryCount: 2

5

consumer-2 처리 → XACK ✓
PEL 비어있음 → at-least-once 완료

XAUTOCLAIM 명령어

```
XAUTOCLAIM kyobo:events issuer-consumers
consumer-2 30000 0-0 COUNT 10
# 30초(30000ms) 이상 idle인 PEL 메시지를
# consumer-2에게 자동 이전 (최대 10개)
```

minIdleMs 설정 기준

- 처리 시간의 **3~5배**를 minIdleMs로 설정
- 너무 짧게 설정 → 처리 중인 메시지가 다른 Consumer에 재배포
- 먹등성이 있으면 안전하지만 불필요한 중복 처리 증가

Redis 7.0+: XAUTOCLAIM 권장

구버전: XPENDING → XCLAIM 조합

XACK 명령어 — 처리 완료 선언

REMOVES FROM PEL — THE ONLY WAY TO DECLARE COMPLETION

기본 형식

```
XACK <stream-key> <group-name> <message-id> [<message-id> ...]
```

단건 ACK

```
XACK kyobo:events issuer-consumers 1714000000000-0
```

결과: (integer) 1 ← ACK된 메시지 수

여러 개 한 번에

```
XACK kyobo:events issuer-consumers 1714000000000-0 1714000001000-0
```

결과: (integer) 2

❌ 잘못된 시점: 처리 시작 전 XACK

XACK → 처리 중 crash → 이벤트 유실
PEL에서 이미 제거됨 → 재전달 없음

❌ 잘못된 시점: 처리와 무관하게 XACK

실제 처리 성공 여부와 XACK가 분리됨

✅ 올바른 시점: 처리 성공 확인 후 XACK

```
try {  
  await processor.process(message);  
  // 성공 후에만 ACK  
  await redis.xack(streamKey, groupName, message.id);  
} catch {  
  // ACK 안 함 → PEL에 남음 → 재시도  
}
```

XACK 명령어 — 실행 흐름

ACK TIMING DETERMINES AT-LEAST-ONCE: PROCESS FIRST, ACK SECOND

잘못된 패턴

XREADGROUP → 메시지 수신
PEL에 등록됨



✗ XACK 먼저 호출
PEL에서 즉시 제거



처리 중 crash
PEL 비어있음 → 재전달 없음
이벤트 영구 유실

올바른 패턴

XREADGROUP → 메시지 수신
PEL: { msg-id: consumer, deliveryCount:1 }



try { await processor.process(msg) }
비즈니스 로직 실행



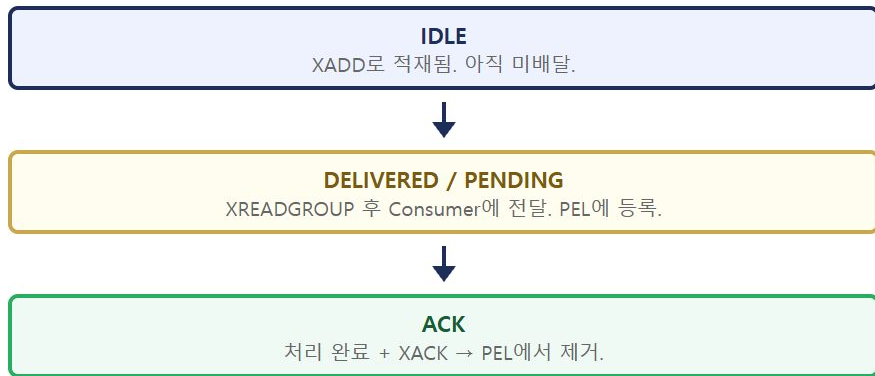
✓ 성공 후 XACK → PEL 제거

catch → ACK 안 함 → PEL에 남음 → 재시도

원칙: 처리 성공 확인 후 XACK — 이것이 At-least-once 보장의 핵심이다.

메시지 상태 3단계 + Redis Streams 핵심 명령 정리

IDLE → DELIVERED/PENDING → ACK · COMMAND REFERENCE



명령	의미
XADD	스트림에 메시지 추가 (발행)
XGROUP CREATE	Consumer Group 생성
XREADGROUP ... >	미처리 새 메시지 수신 + PEL 등록
XACK	처리 완료 선언 — PEL에서 제거
XAUTOCLAIM	idle 초과 PEL 메시지 자동 인계
XPENDING	PEL 조회 (모니터링)

4부 — XRANGE · XREAD · XCLAIM (디버깅/운영)

SUPPLEMENTARY COMMANDS — OUTSIDE MAIN FLOW, ESSENTIAL IN PRODUCTION

4부 · 보완 명령어와 디버깅 도구

XRANGE — ID 범위 직접 조회

```
XRANGE kyobo:events - + # 전체
XRANGE kyobo:events 1714000000-0 + COUNT 10
XREVRANGE kyobo:events + - COUNT 5 # 최신 5개
```

- Consumer Group 없이 스트림 내용 직접 조회
- 시간 복잡도: $O(\log N)$ — 특정 ID 빠르게 검색
- 모니터링·디버깅 필수 명령어

XCLAIM — 수동 소유권 인계

```
XCLAIM kyobo:events issuer-consumers Alice 3600000 171400000000-0
# 1시간 idle 초과 msg를 Alice에게 재배정
```

- Redis 7.0 미만 환경 또는 세밀한 제어 필요 시
- XPENDING 먼저 조회 후 수동 XCLAIM

XREAD — Consumer Group 없는 기본 읽기

```
XREAD COUNT 10 STREAMS kyobo:events 0
XREAD COUNT 1 BLOCK 5000 STREAMS kyobo:events $
```

항목	XREAD	XREADGROUP
Group 필요	없음	있음
메시지 분배	모두 동일 수신	Consumer 간 분담
PEL 관리	없음	있음
장애 복구	없음	XAUTOCLAIM

강의 코드는 **XREADGROUP** 사용 — XREAD는 PEL 없어서 At-least-once 보장 불가

XPENDING · XINFO · XDEL — 운영 모니터링

OBSERVABILITY COMMANDS — KNOW THE STATE OF YOUR PIPELINE

XPENDING 출력 해석

```
XPENDING kyobo:events issuer-consumers
# 1) (integer) 3          ← 전체 pending 수
# 2) "1714000000000-0"    ← 가장 오래된 pending ID
# 3) "1714000003000-0"    ← 가장 최신 pending ID
# 4) consumer-1: 2건, consumer-2: 1건

# 상세 조회 (idle 포함)
XPENDING kyobo:events issuer-consumers IDLE 30000 - + 10
# 각 항목: message-id / consumer / idle-time(ms) / delivery-count
```

XINFO 운영 체크포인트

```
XINFO GROUPS kyobo:events
# pending이 계속 증가 → Consumer 추가 필요
# pending이 0으로 안 떨어짐 → DLQ 확인

XINFO CONSUMERS kyobo:events issuer-consumers
# idle이 비정상적으로 큰 Consumer → crash 의심
# → XAUTOCLAIM 대상
```

XDEL — 메시지 삭제 주의

XDEL 주의: 스트림에서 삭제되어도 PEL에 남음. PEL에 있는 채로 XDEL 시 XREADGROUP이 빈 결과 반환 → XACK 없이 처리된 것처럼 보임.

권장 용도: 개인정보가 담긴 잘못 발행된 메시지 긴급 제거, 개발 환경 테스트 데이터 정리

실습 — Part 1: initialize + publish / Part 2: ConsumerGroupWorker

exercises/S07_redis_stream.ts · npx ts-node src/exercises/S07_redis_stream.ts

Part 1 — RedisStreamPublisher

```
// TODO 1: Consumer Group 생성
await publisher.initialize('issuer-consumers');

// TODO 2: NFT_ISSUED 이벤트 발행
const messageId = await publisher.publish({
  streamKey: 'kyobo:events',
  eventType: 'NFT_ISSUED',
  payload: { to: '0xABCD', tokenId: '42' },
  txHash: '0xdeadbeef',
  blockNumber: 18500001,
  requestId: 'req-s07-001',
});
console.log(`messageId=${messageId}`);

// TODO 3: ping() 연결 확인
const isAlive = await publisher.ping();
if (isAlive) console.log('연결 정상');
```

Part 2 — EventProcessor + ConsumerGroupWorker

```
class SimpleNFTProcessor implements EventProcessor {
  readonly eventTypes = ['NFT_ISSUED'];

  async process(msg: StreamMessage): Promise<void> {
    const p = JSON.parse(msg.fields['payload'] ?? '{}');
    console.log('NFT 처리: tokenId=${p.tokenId}');
  }
}

// TODO 5: Worker 생성 + 1초 실행 후 중지
const worker = new ConsumerGroupWorker({
  redis, config: { streamKey: 'kyobo:events',
    groupName: 'issuer-consumers', consumerId: 's07-consumer',
    batchSize: 5, blockMs: 100, minIdleMs: 5_000 },
  processors: [processor], dlq: dlqHandler,
});
worker.start();
await new Promise(r => setTimeout(r, 1000));
worker.stop();
```

실습 코드 — 실행 흐름

PART 1: INIT → PUBLISH → PING / PART 2: PROCESSOR → WORKER LIFECYCLE

Part 1 — RedisStreamPublisher

① publisher.initialize('issuer-consumers')

XGROUP CREATE kyobo:events ... \$ · BUSYGROUP 무시



② publisher.publish(event)

payload→JSON · blockNumber→String · XADD → messageId



③ publisher.ping()

redis.ping() → true / catch → false

Part 2 — ConsumerGroupWorker

① SimpleNFTProcessor 구현

eventTypes=['NFT_ISSUED'] · process() → JSON.parse → log



② new ConsumerGroupWorker({...})

batchSize=5 · blockMs=100 · minIdleMs=5000



③ worker.start() → setTimeout(1000) → worker.stop()

XREADGROUP 루프 1초 실행 후 정지

핵심: Part 1은 발행자 생명주기(초기화→발행→상태확인), Part 2는 소비자 생명주기(정의→생성→실행→종료)를 각각 실습한다.

핵심 정리 · 예상 Q&A · 다음 세션 예고

KEY CONCEPTS · ANTICIPATED QUESTIONS · NEXT SESSION

패턴	보장 수준	사용 조건
Pub/Sub	없음 (fire-and-forget)	유실 허용 가능한 알림
List Queue	at-most-once	단순 작업 큐
Redis Streams + Group	at-least-once	금융 이벤트 처리
Streams + 멍등성	effectively exactly-once	금융 원장 업데이트

S8 실습 예고: Docker 기동 + XADD 3개 적재 → XGROUP CREATE → XREADGROUP → XACK + XPENDING PEL 변화 확인 → XAUTOCLAIM 미처리 재수신 시뮬레이션 → Consumer 2개 분배 확인

Q1. Redis Streams가 Kafka와 무슨 차이인가?

개념 동일 (Consumer Group, PEL, offset, XACK). 차이: Kafka는 파티션 단위 순서 보장 · 디스크 영구 보존 · 수백만 TPS. 교보 시스템 규모(하루 수천 건)에선 Redis Streams로 충분.

Q2. XACK를 처리 성공 전에 먼저 보내면?

→ S9에서 케이스별로 상세히 다룬다. 단, 결론: PEL에서 즉시 제거 → crash 시 재전달 없음 → 이벤트 유실.

Q3. minIdleMs를 너무 짧게 설정하면?

처리 중인 메시지가 다른 Consumer에 재배분됨. LedgerService.update()가 5초 걸리는데 minIdleMs=3000이면 처리 완료 전 중복 처리. 처리 시간의 3~5배 권장.